

DrugGuard - Diabetic Drug Interaction Checker

Complete Project Documentation (Updated January 2026)

Table of Contents

- [1. Project Overview](#)
- [2. System Architecture](#)
- [3. Core Features](#)
- [4. Hybrid Decision Engine](#)
- [5. Machine Learning Pipeline](#)
- [6. LLM Integration](#)
- [7. OCR & Prescription Scanning](#)
- [8. Prescription RAG Chat](#)
- [9. Diabetic Patient Management](#)
- [10. Frontend & Mobile App](#)
- [11. API Reference](#)
- [12. Technology Stack](#)
- [13. Installation & Setup](#)
- [14. Data Sources](#)

1. Project Overview

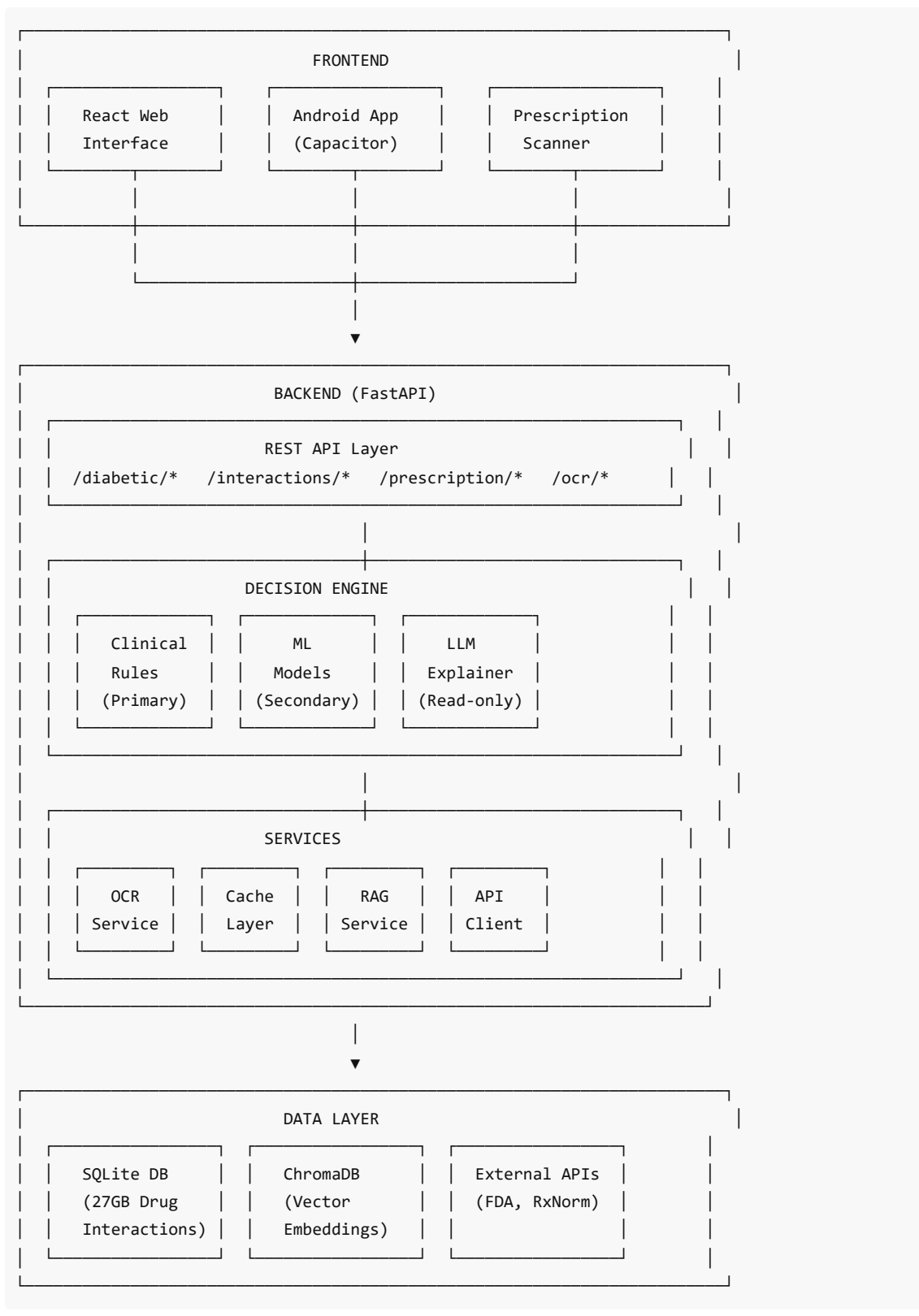
DrugGuard is a comprehensive clinical decision support system designed specifically for **diabetic patients**. It combines evidence-based clinical rules with machine learning to:

- ✓ Check drug-drug interactions
- ✓ Assess drug safety for diabetic patients
- ✓ Scan prescriptions using camera/OCR
- ✓ Provide patient-friendly explanations via LLM
- ✓ Suggest safer alternatives
- ✓ Chat with prescriptions using RAG

Key Differentiators

Traditional Drug Checkers	DrugGuard
Generic interaction lookup	Diabetes-specific risk assessment
No patient context	Considers eGFR, complications, comorbidities
Text-only input	Camera/OCR prescription scanning
No explanations	LLM-powered patient-friendly explanations
Web only	Android app available

2. System Architecture



3. Core Features

3.1 Drug-Drug Interaction Checking

How it works:

- 1. User enters two drug names (text or search)
- 2. System checks 27GB database for known interactions
- 3. Clinical rules evaluate severity
- 4. ML models predict unknown interactions
- 5. Results displayed with recommendations

Severity Levels:

Level	Icon	Description
Safe		No known interaction
Minor		Generally safe, minimal effect
Moderate		Use caution, monitor
Major		Significant risk, consult doctor
Contraindicated		Do NOT use together

3.2 Safe Alternatives

When an interaction is found, the system automatically suggests:

- Drugs in the same therapeutic class
- Drugs that don't interact with the other medication
- Ranked by safety profile

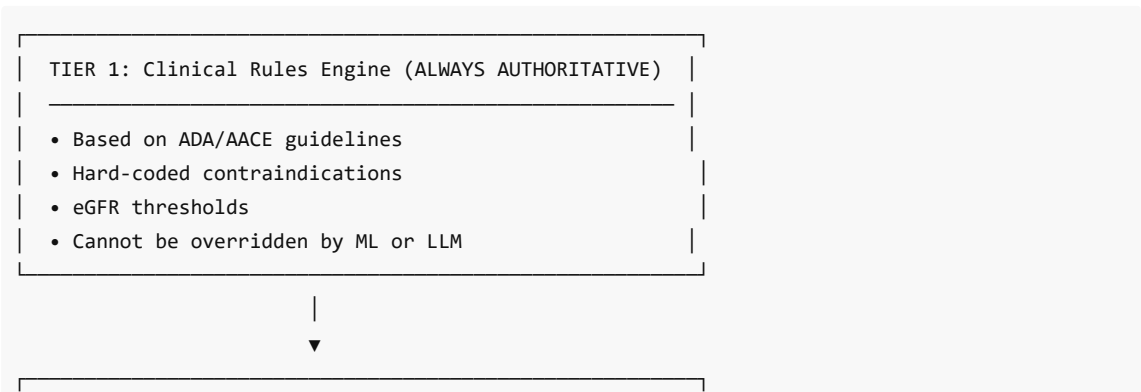
3.3 Diabetic Patient Risk Assessment

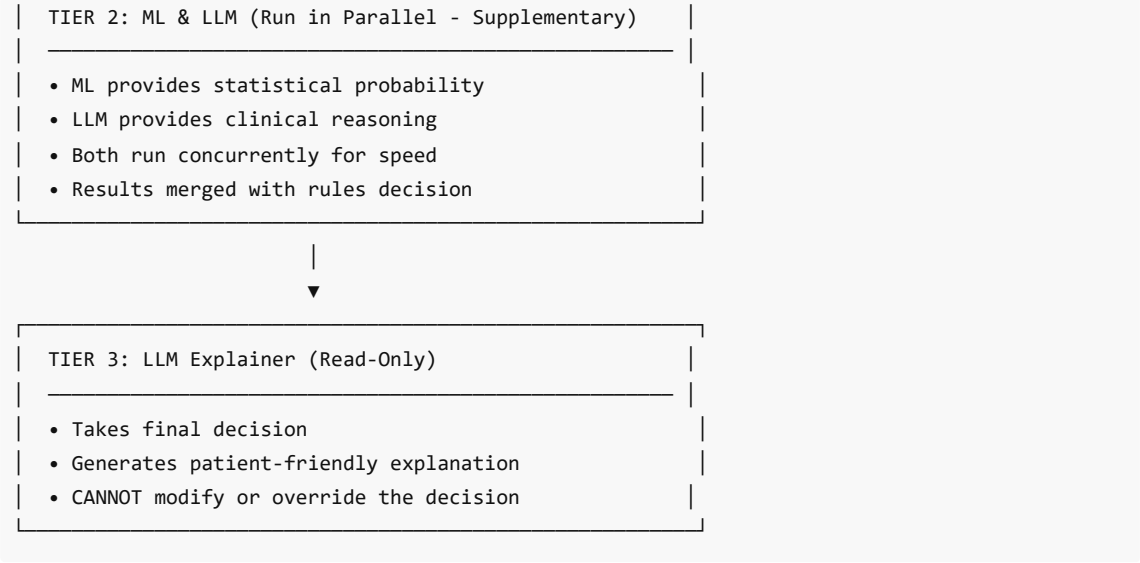
Unique diabetes-specific checks:

- **eGFR-based dosing** - Adjusts recommendations based on kidney function
- **Hypoglycemia risk** - Flags drugs that mask low blood sugar
- **Nephrotoxicity** - Warns about kidney-damaging drugs
- **Dangerous combinations** - Detects "Triple Whammy" and similar patterns
- **Complication awareness** - Considers nephropathy, retinopathy, CVD

4. Hybrid Decision Engine

DrugGuard uses a **three-tier hybrid architecture** where safety always comes first:





Decision Logic:

```
# High-risk (Contraindicated/Major): ALWAYS use rules
if rule_severity in ["contraindicated", "major", "fatal"]:
    final_decision = rule_result
    source = "rule_override"

# Moderate/Unknown: Consider ML prediction
elif ml_probability is not None:
    final_decision = combine(rule_result, ml_result)
    source = "ml_primary"

# Fallback: Use rules if ML unavailable
else:
    final_decision = rule_result
    source = "rules_only"
```

5. Machine Learning Pipeline

5.1 Models Used

Model	Purpose	Performance
XGBoost	Primary predictor	Fast, accurate
Random Forest	Ensemble diversity	Robust
LightGBM	Imbalanced data	High recall

5.2 Training Data

- **TWOSIDES Database:** 2M+ drug-drug interaction records
- **Features per drug pair:** 242 dimensions
 - Drug class encoding (100 features)

- Mechanism hashing (60 features)
- Name hashing (80 features)
- Boolean flags (2 features)

5.3 Key Metrics

Metric	Value	Importance
NPV	99.41%	Critical - minimize false negatives
Sensitivity	85%+	Catch real interactions
Specificity	90%+	Avoid false alarms

5.4 Ensemble Prediction

```
# Each model predicts probability
rf_prob = random_forest.predict_proba(X)[0, 1]
xgb_prob = xgboost.predict_proba(X)[0, 1]
lgb_prob = lightgbm.predict_proba(X)[0, 1]

# Average for final prediction
ensemble_prob = (rf_prob + xgb_prob + lgb_prob) / 3

# Map to severity
if ensemble_prob >= 0.9: severity = "contraindicated"
elif ensemble_prob >= 0.7: severity = "major"
elif ensemble_prob >= 0.4: severity = "moderate"
elif ensemble_prob >= 0.2: severity = "minor"
else: severity = "none"
```

6. LLM Integration

DrugGuard uses LLMs in **three distinct ways**:

6.1 LLM Explainer (llm_explainer.py)

Purpose: Generate patient-friendly explanations of risk assessments.

Safety Rules (from system prompt):

1. ONLY explains decisions already made by rules
2. NEVER recommends changing medications
3. NEVER contradicts the risk assessment
4. ALWAYS recommends consulting healthcare provider
5. Keeps explanations under 100 words

Example Output:

```
"Your doctor has flagged Metformin for monitoring because of your
kidney function. While Metformin is generally safe, it's processed
```

by your kidneys, so we need to keep an eye on them. Please don't skip your checkups!"

6.2 LLM Drug Checker (llm_drug_checker.py)

Purpose: Provide clinical reasoning analysis (runs in parallel with ML).

Example Output (JSON):

```
{
  "risk_level": "high_risk",
  "risk_score": 75,
  "reasoning": "Patient has eGFR 40 (Stage 3b CKD). NSAIDs are
               nephrotoxic. Risk of Triple Whammy with ACE inhibitor.",
  "key_concerns": ["Nephrotoxicity", "Triple whammy risk"],
  "monitoring_needed": ["Renal function", "Blood pressure"]
}
```

6.3 Prescription RAG Chat (rag_service.py)

Purpose: Answer questions about uploaded prescriptions.

Flow:

- 1. Prescription indexed in ChromaDB (vector embeddings)
- 2. User asks question
- 3. Semantic search retrieves relevant context
- 4. LLM generates answer based ONLY on prescription content

Supported LLMs (with fallback chain):

- 1. **Gemini** (gemini-1.5-flash) - if API key available
- 2. **Ollama** (gpt-oss:120b-cloud / llama3.1:8b) - local
- 3. **Template Engine** - if all LLMs fail

7. OCR & Prescription Scanning

7.1 Technology Stack

Component	Technology
OCR Engine	Tesseract
Image Processing	OpenCV
Name Matching	RapidFuzz

7.2 Pipeline

Image Input → Preprocessing → Multiple Threshold Passes → OCR →
Pattern Matching → Fuzzy Name Matching → Validated Drug List

7.3 Image Preprocessing Steps

- 1. Convert to grayscale
- 2. Apply adaptive thresholding
- 3. Deskew rotated images
- 4. Bilateral filter (noise reduction)
- 5. Run OCR on multiple preprocessed versions
- 6. Merge and deduplicate results

7.4 Drug Name Recognition

- Pattern matching for common drug name formats
- Fuzzy matching against 27GB database
- Handles OCR errors and partial matches
- Confidence scoring

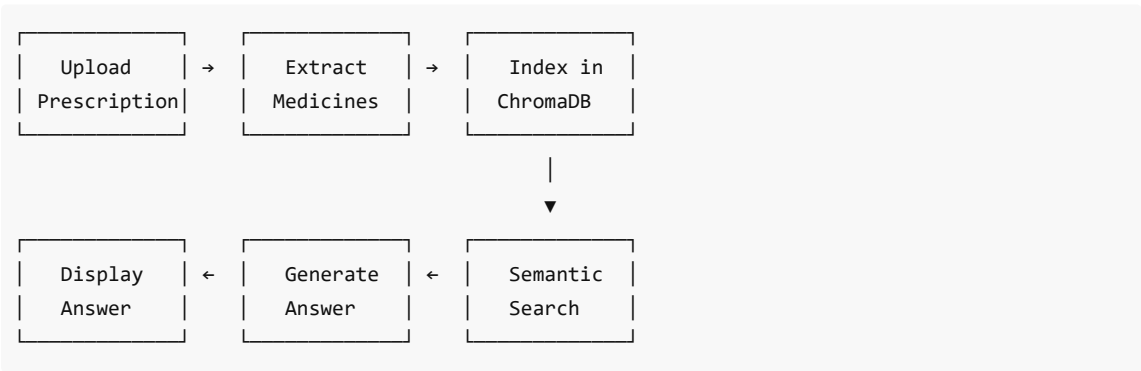
8. Prescription RAG Chat

8.1 What is RAG?

Retrieval-Augmented Generation - Combines:

- **Retrieval:** Search prescription content semantically
- **Generation:** LLM generates natural language answers

8.2 How It Works



8.3 Example Chat Session

User: "When should I take Nucoxia-MR?"

System retrieves:
Medicine: Nucoxia-MR
Dosage: 500mg
Frequency: Twice daily
When to take: morning, night

Answer: "Based on your prescription, you should take Nucoxia-MR 500mg twice daily - once in the morning and once at night. Take it after meals."

9. Diabetic Patient Management

9.1 Patient Profile

Stored Patient Data:

```
{
  "patient_id": "DEMO001",
  "age": 65,
  "gender": "M",
  "diabetes_type": "type_2",
  "labs": {
    "egfr": 45,
    "hba1c": 7.2,
    "creatinine": 1.4,
    "potassium": 4.8
  },
  "complications": ["nephropathy", "retinopathy"],
  "current_medications": ["Metformin", "Lisinopril", "Insulin"]
}
```

9.2 Risk Assessment Factors

Factor	What It Checks
eGFR	Kidney function thresholds
HbA1c	Diabetes control level
Nephropathy	Avoid nephrotoxic drugs
Retinopathy	Blood pressure medication caution
Cardiovascular	CV-safe drug selection
Current Meds	Drug-drug interactions

9.3 Clinical Rules Examples

eGFR-Based Contraindications:

```
EGFR_THRESHOLDS = {
  "metformin": {"contraindicated_below": 30, "caution_below": 45},
  "nsaid": {"caution_below": 60},
  "lithium": {"caution_below": 60},
}
```

Dangerous Combinations:

```
CONTRAINDICATED_COMBINATIONS = {
  "metformin": ["contrast_dye", "alcohol"],
  "sglit2_inhibitors": ["loop_diuretics"],
}
```



```
}

# "Triple Whammy" - High nephrotoxicity risk
TRIPLE_WHAMMY = ["NSAID", "ACE_inhibitor", "Diuretic"]
```

10. Frontend & Mobile App

10.1 Web Application

Technology Stack:

- React 18 with hooks
- Vite (build tool)
- TailwindCSS (styling)
- Framer Motion (animations)

Key Components:

Component	Purpose
InteractionChecker.jsx	Main drug checking UI
DiabetesManager.jsx	Patient profile management
PrescriptionScanner.jsx	Camera capture for OCR
ResultsDisplay.jsx	Interaction results
ModelDashboard.jsx	ML model metrics

10.2 Android App

Built with: Capacitor (React → Native)

APK: DrugGuard.apk (7.3 MB)

Features:

- Camera access for prescription scanning
- Offline caching of common lookups
- Native file picker
- All web features available

11. API Reference

11.1 Diabetic Patient Endpoints

Endpoint	Method	Description
/diabetic/patients	POST	Create patient profile
/diabetic/patients/{id}	GET	Get patient details
/diabetic/risk-check	POST	Check drug safety for patient

/diabetic/alternatives	POST	Get safer alternatives
------------------------	------	------------------------

11.2 General Drug Interaction Endpoints

Endpoint	Method	Description
/interactions/check	POST	Check drug-drug interaction
/interactions/{drug}	GET	Get all interactions for drug
/drugs/search	GET	Search drugs by name
/alternatives	POST	Find safe alternatives

11.3 OCR Endpoints

Endpoint	Method	Description
/ocr/extract	POST	Extract drugs from image
/ocr/upload	POST	Upload medication label

11.4 Prescription RAG Endpoints

Endpoint	Method	Description
/prescription/upload	POST	Upload prescription
/prescription/chat	POST	Ask about prescription
/prescription/{id}	GET	Get prescription details
/prescription/history	GET	Get chat history

11.5 Health & Monitoring

Endpoint	Method	Description
/health	GET	API health check
/health/apis	GET	External API status
/ml/metrics	GET	ML model performance

12. Technology Stack

Backend

Technology	Purpose
FastAPI	Web framework
SQLAlchemy	ORM

SQLite	Database (27GB drug data)
ChromaDB	Vector database (RAG)
Ollama	Local LLM inference
Tesseract	OCR engine
OpenCV	Image processing

ML/AI

Technology	Purpose
XGBoost	Gradient boosting
LightGBM	Gradient boosting
Scikit-learn	Random Forest, utilities
Sentence-Transformers	Embeddings for RAG
Ollama/Gemini	LLM inference

Frontend

Technology	Purpose
React 18	UI framework
Vite	Build tool
TailwindCSS	Styling
Capacitor	Android packaging
Framer Motion	Animations

13. Installation & Setup

Prerequisites

- Python 3.10+
- Node.js 18+
- Tesseract OCR
- Ollama (optional, for LLM)

Quick Start

```
# Clone repository
git clone https://github.com/Dhritimanmitraa/diabetic-ddi.git
cd diabetic-ddi
```

```
# Start application (Windows)
.\run_app.bat
```

Manual Setup

Backend:

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Access Points

Service	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000
API Docs	http://localhost:8000/docs

14. Data Sources

Source	Content	Usage
TWOSIDES	2M+ drug interactions	Training data
OpenFDA	Drug labels, adverse events	Real-time lookup
RxNorm	Drug name normalization	Name matching
ADA/AACE	Clinical guidelines	Rules engine
DrugBank	Drug information	Alternatives

Summary

DrugGuard is a comprehensive, safety-first drug interaction checker designed for diabetic patients, featuring:

- ✔ Hybrid Decision Engine - Rules + ML + LLM
- ✔ Diabetes-Specific - eGFR, complications, hypoglycemia risk
- ✔ Camera Scanning - OCR for prescriptions
- ✔ RAG Chat - Ask questions about your prescription

- ✓ **Patient-Friendly** - LLM-powered explanations
 - ✓ **Mobile Ready** - Android app available
 - ✓ **27GB Database** - 2M+ known interactions
 - ✓ **Safety First** - Clinical rules always override ML
-

Author: Dhritiman Mitra

License: MIT

Last Updated: January 2026